HDOC Day-48

Name- Aaryan Pratap Singh

Sap id- 590032842

Algorithm

1. Start.
2. Read the size `n` of the array.
3. Check that `n` is positive. If not, ask for the size again.
4. Declare a variable-length integer array `arr[n]`.
5. Read the elements of the array.
6. Pass the array to `pivot_index()` using its address.
7. Inside `pivot_index()`:
    - Calculate the sum of all array elements and store it in `totalSum`.
    - Initialize `leftSum = 0`.
    - For each index `i` from `0` to `n-1`:
        - Calculate `rightSum = totalSum - leftSum - *(arr + i)`.
        - Compare `leftSum` and `rightSum`.
        - If they are equal, display `i` as the pivot index and return immediately. This ensures the **leftmost** pivot is displayed.
        - Add the current element to `leftSum`.
8. If no index satisfies the condition, display `-1`.
9. Stop.

Test-Case Table

Test	Array	Expected Output	Reason
1	`1 7 3 6 5 6`	`Pivot index: 3`	Left sum = 11, Right sum = 11
2	`1 2 3`	`-1`	No pivot exists
3	`2 1 -1`	`Pivot index: 0`	Left sum = 0, Right sum = 0
4	`1`	`Pivot index: 0`	Both sums are 0
5	`1 -1 0`	`Pivot index: 2`	Left sum = 0, Right sum = 0
6	`0 0 0`	`Pivot index: 0`	Leftmost pivot is index 0
7	`-1 -1 -1 -1 -1 0`	`Pivot index: 2`	Left sum = -2, Right sum = -2

C Program

```
#include <stdio.h>

void pivot_index(int *arr, int n)

{

   int totalSum = 0;

   int leftSum = 0;

   int rightSum;

   // Calculate total sum of the array

   for (int i = 0; i < n; i++)

   {

      totalSum += *(arr + i);

   }

   // Find the leftmost pivot index

   for (int i = 0; i < n; i++)

   {

      rightSum = totalSum - leftSum - *(arr + i);

      if (leftSum == rightSum)

      {

         printf("Pivot index: %d\n", i);

         return;

      }

      leftSum += *(arr + i);
```

```
    }
    // No pivot index found
    printf("-1\n");
}
int main()
{
    int n;
    printf("Enter the size of the array: ");
    scanf("%d", &n);
    while (n <= 0)
    {
        printf("Invalid size. Enter a positive size: ");
        scanf("%d", &n);
    }
    // Variable-length array
    int arr[n];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", arr + i);
    }
    // Pass array using its address
```

```
    pivot_index(arr, n);

    return 0;

}
```

EXECUTION

```
[aaryanrajput@aaryans-macbook ~ % nano day48.c
[aaryanrajput@aaryans-macbook ~ % clang day48.c -o day48.c
[aaryanrajput@aaryans-macbook ~ % ./day48.c
 Enter the size of the array: 6
 Enter 6 elements:
 1 7 3 6 5 6
 Pivot index: 3
[aaryanrajput@aaryans-macbook ~ % ./day48.c
 Enter the size of the array: 3
 Enter 3 elements:
 1 2 3
 -1
[aaryanrajput@aaryans-macbook ~ % ./day48.c
 Enter the size of the array: 3
 Enter 3 elements:
 2 1 -1
 Pivot index: 0
 aaryanrajput@aaryans-macbook ~ %
```